ASSIGNMENT - 3

NUMERICAL TECHNIQUES IN HYDRODYNAMICS.

By :-
J. Karthick.
OE23S018

Problem statement :-

Solve the poisson's equations using the Boundary condition and sources. Test the solution for the different values of A = 0.05, 0.01, 0.1.

Aim :-

To solve the poisson's equations and varying the values of A under the Jacobi Iteration, Gauss Siedel, Successive Under Relaxation methods.

Poissons equation :-

$$\frac{\partial^2 T}{\partial x^2} + \frac{\partial^2 T}{\partial y^2} = Q(x,y).$$

Boundary conditions :-

Case : 1

$T = 0$ at $y = 0$ and $y = 1.0$

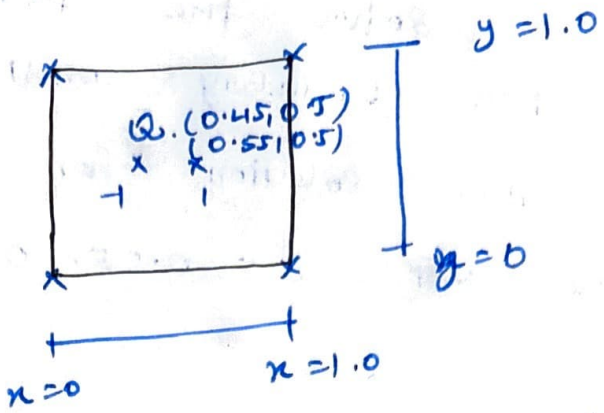
$T = 10^\circ$ at $x = 0$ and $x = 1.0$

Case : 2.

$T = 0$ at $y = 0$ and $y = 1.0$

$T = 0^\circ$ at $x = 0$ and $\frac{\partial T}{\partial x} = -1.0$ at $x = 1.0$.

Schematic 2D Heat solution :-



Finite difference method For solution :-

Governing (or) general poissons equations :-

$$\frac{\partial^2 T}{\partial x^2} + \frac{\partial^2 T}{\partial y^2} = Q(x,y).$$

using the central difference method

$$\frac{T_{i-1j} + T_{i+1j} - 2T_{ij}}{\Delta x^2} + \frac{T_{ij-1} + T_{ij+1} - 2T_{ij}}{\Delta y^2} = Q(x,y).$$

Jacobi Iteration method :-

by solving the above equation and the uniform mesh $\Delta x = \Delta y = \Delta$

$$T_{ij}^{n+1} = \frac{1}{4}\left[\left[T_{i+1j}^{n} + T_{i-1j}^{n} + T_{ij-1}^{n} + T_{ij+1}^{n}\right] - \Delta x^2 [Q_{ij}].\right]$$

Gauss Siedel Iteration method :-

$$T_{ij}^{n+1} = \frac{1}{4}\left[T_{i+1j}^{n+1} + T_{i+1j}^{n} + T_{ij-1}^{n+1} + T_{ij+1}^{n} - \Delta x^2 (Q_{ij}) \right]$$

Successive under Relaxation :-

$$T_{ij}^{n+1} = \frac{1}{4}\left[T_{ij}^{n+1} + T_{i+1j}^{n} + T_{ij-1}^{n+1} + T_{ij+1}^{n} - \Delta x^2 Q(x,y) \right]$$

under Relaxation.

$$T_{ij}^{n+1} = T_{ij}^{n} + \alpha \left[T_{ij}^{n+1} - T_{ij}^{n} \right].$$

S.NO	BOUNDARY CONTIONS CASES	ITERATIVE METHOD	DELTA	Number of ITERTION
1	CASE 1	JACOBI	0.1	185
		GAUSS SIEDEL	0.1	91
		SUCCESSIVE UNDER RELAXATIOM	0.1	165
2	CASE 1	JACOBI	0.05	559
		GAUSS SIEDEL	0.05	308
		SUCCESSIVE UNDER RELAXATIOM	0.05	553
3	CASE 1	JACOBI	0.01	7491
		GAUSS SIEDEL	0.01	4441
		SUCCESSIVE UNDER RELAXATIOM	0.01	7753
4	CASE 2	JACOBI	0.1	166
		GAUSS SIEDEL	0.1	149
		SUCCESSIVE UNDER RELAXATIOM	0.1	264
5	CASE 2	JACOBI	0.05	558
		GAUSS SIEDEL	0.05	496
		SUCCESSIVE UNDER RELAXATIOM	0.05	883
6	CASE 2	JACOBI	0.01	7474
		GAUSS SIEDEL	0.01	7081
		SUCCESSIVE UNDER RELAXATIOM	0.01	12345

Algorithm: Jacobi Iteration for Poisson's Equation

Input Parameters:

- Domain dimensions: Lx,LyLx,Ly
- Grid spacing: Δx,ΔyΔx,Δy
- Number of grid points: MX,NYMX,NY
- Error tolerance: ERROR$ERROR$
- Maximum iterations: Max_iteration$Max_iteration$
- Boundary conditions: Tmax,Tmin$Tmax$,$Tmin$

Initialization:

1. **Grid Setup:**
    - Calculate MX=LxΔx+1$MX=\frac{Lx}{\Delta x}+1$ and NY=LyΔy+1$NY=\frac{Ly}{\Delta y}+1$.
    - Initialize temperature array T[i][j]=0$T[i][j]=0$ for all i,ji,j.
2. **Boundary Conditions:**
    - Apply boundary conditions to T[i][j]$T[i][j]$:
        - Top: T[i][NY−1]=Tmax$T[i][NY-1]=Tmax$
        - Bottom: T[i]=Tmax$T[i]=Tmax$
        - Left: T[j]=Tmin$T[j]=Tmin$
        - Right: T[MX−1][j]=Tmin$T[MX-1][j]=Tmin$

Jacobi Iteration Loop:

1. **Iteration Counter:**
    - Initialize iteration counter: iter=0$iter=0$.
2. **Maximum Difference:**
    - Initialize maximum difference: max_diff=ERROR$max_diff=ERROR$.
3. **Loop Until Convergence:**
    - While max_diff>ERROR$max_diff>ERROR$ and iter<Max_iteration $iter<Max_iteration$:
        1. **Update Temperature Array:**

- For each interior grid point (i,j)(i,j):
    - Calculate new temperature Tnew[i][j]$Tnew[i][j]$ using:

Tnew[i][j]=14(T[i+1][j]+T[i−1][j]+T[i][j+1]+T[i][j−1]−(Δx)2fi,j)$Tnew[i][j]=41(T[i+1][j]+T[i-1][j]+T[i][j+1]+T[i][j-1]-(\Delta x)2fi,j)$

    - Note: fi,jfi,j is a source term, if applicable.

2. **Calculate Error:**
    - For each interior grid point (i,j)(i,j):
        - Calculate absolute difference between new and old values: error=|Tnew[i][j]−T[i][j]|$error=|Tnew[i][j]-T[i][j]|$
        - Update max_diffmax_diff if error>max_diff$error>max_diff$
3. **Update Temperature Array:**
    - For each interior grid point (i,j)(i,j):
        - Update T[i][j]=Tnew[i][j]$T[i][j]=Tnew[i][j]$
4. **Increment Iteration Counter:**
    - iter++$iter++$
5. **Write Grid to File (Optional):**
    - If desired, write the current temperature grid to a file for visualization.

Convergence Check:

- If max_diff≤ERROR$max_diff \leq ERROR$ or iter≥Max_iteration$iter \geq Max_iteration$, exit loop.

Final Output:

- Write final temperature grid to file.
- Print convergence status.

Algorithm: Gauss-Seidel Iteration for Poisson's Equation

Input Parameters:

- Domain dimensions: Lx,LyLx,Ly
- Grid spacing: Δx,ΔyΔx,Δy
- Number of grid points: MX,NYMX,NY
- Error tolerance: ERROR$ERROR$
- Maximum iterations: Max_iteration$Max_iteration$
- Boundary conditions: Tmax,Tmin$Tmax$,$Tmin$

Initialization:

1. **Grid Setup:**
    - Calculate MX=LxΔx+1$MX=\frac{Lx}{\Delta x}+1$ and NY=LyΔy+1$NY=\frac{Ly}{\Delta y}+1$.
    - Initialize temperature array T[i][j]=0$T[i][j]=0$ for all i,ji,j.
2. **Boundary Conditions:**
    - Apply boundary conditions to T[i][j]$T[i][j]$:
        - Top: T[i][NY−1]=Tmax$T[i][NY-1]=Tmax$
        - Bottom: T[i]=Tmax$T[i]=Tmax$
        - Left: T[j]=Tmin$T[j]=Tmin$
        - Right: T[MX−1][j]=Tmin$T[MX-1][j]=Tmin$

Gauss-Seidel Iteration Loop:

1. **Iteration Counter:**
    - Initialize iteration counter: iter=0$iter=0$.
2. **Maximum Difference:**
    - Initialize maximum difference: max_diff=0$max_diff=0$.
3. **Loop Until Convergence:**
    - While max_diff>ERROR$max_diff>ERROR$ and iter<Max_iteration $iter<Max_iteration$:
        1. **Update Temperature Array:**
            - For each interior grid point (i,j)(i,j):

- Calculate new temperature Tnew[i][j]$Tnew[i][j]$ using:

Tnew[i][j]=14(T[i+1][j]+T[i−1][j]+T[i][j+1]+T[i][j−1]−(Δx)2fi,j)$Tnew[i][j]=41(T[i+1][j]+T[i-1][j]+T[i][j+1]+T[i][j-1]-(\Delta x)2fi,j)$

- Note: fi,jfi,j is a source term, if applicable.

2. **Calculate Error:**
   - For each interior grid point (i,j)(i,j):
     - Calculate absolute difference between new and old values: error=|Tnew[i][j]−T[i][j]|$error=|Tnew[i][j]-T[i][j]|$
     - Update max_diffmax_diff if error>max_diff$error>max_diff$
3. **Update Temperature Array:**
   - For each interior grid point (i,j)(i,j):
     - Update T[i][j]=Tnew[i][j]$T[i][j]=Tnew[i][j]$ immediately for Gauss-Seidel.
4. **Increment Iteration Counter:**
   - iter++$iter++$
5. **Write Grid to File (Optional):**
   - If desired, write the current temperature grid to a file for visualization.

Convergence Check:

- If max_diff≤ERROR$max_diff \leq ERROR$ or iter≥Max_iteration$iter \geq Max_iteration$, exit loop.

Final Output:

- Write final temperature grid to file.
- Print convergence status.

Algorithm: Successive Under-Relaxation for Poisson's Equation

Input Parameters:

- Domain dimensions: Lx,LyLx,Ly
- Grid spacing: Δx,ΔyΔx,Δy
- Number of grid points: MX,NYMX,NY
- Error tolerance: ERROR$ERROR$
- Maximum iterations: Max_iteration$Max_iteration$
- Boundary conditions: Tmax,Tmin$Tmax$,$Tmin$
- Under-relaxation factor: αα

Initialization:

1. **Grid Setup:**
    - Calculate MX=LxΔx+1$MX=\frac{Lx}{\Delta x}+1$ and NY=LyΔy+1$NY=\frac{Ly}{\Delta y}+1$.
    - Initialize temperature array T[i][j]=0$T[i][j]=0$ for all i,ji,j.
2. **Boundary Conditions:**
    - Apply boundary conditions to T[i][j]$T[i][j]$:
        - Top: T[i][NY−1]=Tmax$T[i][NY-1]=Tmax$
        - Bottom: T[i]=Tmax$T[i]=Tmax$
        - Left: T[j]=Tmin$T[j]=Tmin$
        - Right: T[MX−1][j]=Tmin$T[MX-1][j]=Tmin$

Successive Under-Relaxation Iteration Loop:

1. **Iteration Counter:**
    - Initialize iteration counter: iter=0$iter=0$.
2. **Maximum Difference:**
    - Initialize maximum difference: max_diff=0$max_diff=0$.
3. **Loop Until Convergence:**
    - While max_diff>ERROR$max_diff>ERROR$ and iter<Max_iteration $iter<Max_iteration$:

1. **Update Temperature Array:**
    - For each interior grid point (i,j)(i,j):
        - Calculate new temperature Tnew[i][j]$Tnew[i][j]$ using:

Tnew[i][j]=14(T[i+1][j]+T[i−1][j]+T[i][j+1]+T[i][j−1]−(Δx)2fi,j)$Tnew[i][j]=41(T[i+1][j]+T[i−1][j]+T[i][j+1]+T[i][j−1]−(\Delta x)2fi,j)$

        - Note: fi,jfi,j is a source term, if applicable.
2. **Apply Under-Relaxation:**
    - Update T[i][j]$T[i][j]$ using under-relaxation:

T[i][j]=T[i][j]+α(Tnew[i][j]−T[i][j])$T[i][j]=T[i][j]+\alpha(Tnew[i][j]−T[i][j])$

3. **Calculate Error:**
    - For each interior grid point (i,j)(i,j):
        - Calculate absolute difference between new and old values: error=|Tnew[i][j]−T[i][j]|$error=|Tnew[i][j]−T[i][j]|$
        - Update max_diffmax_diff if error>max_diff$error>max_diff$
4. **Increment Iteration Counter:**
    - iter++$iter++$
5. **Write Grid to File (Optional):**
    - If desired, write the current temperature grid to a file for visualization.

Convergence Check:

- If max_diff≤ERROR$max_diff≤ERROR$ or iter≥Max_iteration$iter≥Max_iteration$, exit loop.

Final Output:

- Write final temperature grid to file.
- Print convergence status.

Code For Jacobi iteration Method:

```
//********************JACOBI ITERATION METHOD*********************//
#include <stdio.h>
#include <stdlib.h>
#include <math.h>

#define L 1.0 //x and y axis length of the domain
#define delta_x 0.01 //length of the cell size
#define delta_y 0.01 //length of the cell size
#define MX ((int)(L/delta_x)+1) // number of cells in x axis
#define NY ((int)(L/delta_y)+1) // number of cells in y axis
#define ERROR 0.0001 //error for the iteration
#define T_max 10.0 //maximum temperature of the boundary condition in degrees
#define T_min 0.0 //Minimum temperature of the boundary condition in degrees
#define Max_iteration 10000 //maximum iteration for the convergence

float T[MX][NY], T_new[MX][NY];

// Function to set boundary conditions
void setBoundaryConditions() {
    int i, j;
    // Top side Boundary condition
    for (i = 0; i < MX; i++) {
        T[i][NY-1] = T_max;
    }
    // Bottom side Boundary condition
```

```
    for (i = 0; i < MX; i++) {
        T[i][0] = T_max;
    }
    // Left side Boundary condition
    for (j = 0; j < NY; j++) {
        T[0][j] = T_min;
    }
    // Right side Boundary condition
    for (j = 0; j < NY; j++) {
        T[MX-1][NY]= T_min;
    }
}

// Function to perform Jacobi iteration poisson equation
void jacobiIteration() {
    int i, j;
    float q1 = 1.0, q2 = -1.0;
    int sx1 = (int)(0.45 / delta_x), sy1 = (int)(0.5 / delta_y);
    int sx2 = (int)(0.55 / delta_x), sy2 = (int)(0.5 / delta_y);

    for (i = 1; i < MX-1; i++) {
        for (j = 1; j < NY-1; j++) {
            float source = 0;
            if (i == sx1 && j == sy1) source = q1;
            if (i == sx2 && j == sy2) source = q2;
            T_new[i][j] = 0.25 * (T[i+1][j] + T[i-1][j] + T[i][j+1] + T[i][j-1] -
(delta_x * delta_x) * source);
        }
```

```
    }
}

// Function to write grid to file in gnuplot format
void writeGridToFile(FILE *file) {
    for (int i = 0; i < MX; i++) {
        for (int j = 0; j < NY; j++) {
            fprintf(file, "%f ", T[i][j]);
        }
        fprintf(file, "\n");
    }
    fprintf(file, "\n\n"); // Two blank lines between datasets for gnuplot
}

int main() {
    float max_diff;
    int iter = 0;

    printf("Number of cells in the X-axis: %d\n", MX);
    printf("Number of cells in the Y-axis: %d\n", NY);

    // Initialize temperature array
    for (int i = 0; i < MX; i++) {
        for (int j = 0; j < NY; j++) {
            T[i][j] = 0.0;
        }
    }
```

```
// Set boundary conditions
setBoundaryConditions();

// Open output file
FILE *outputFile = fopen("temperature_field.txt", "w");
if (outputFile == NULL) {
    printf("Error opening output file!\n");
    return 1;
}

// Jacobi iteration
do {
    jacobiIteration();

    // Calculate error and convergence criteria
    max_diff = ERROR;
    for (int i = 1; i < MX-1; i++) {
        for (int j = 1; j < NY-1; j++) {
            float error = fabs(T_new[i][j] - T[i][j]);
            if (error > max_diff) {
                max_diff = error;
            }
            T[i][j] = T_new[i][j];
        }
    }
```

```
        iter++;

        // Write grid to file every 1 iterations
        if (iter % 1 == 0) {
            writeGridToFile(outputFile);
        }

    } while (max_diff > ERROR && iter < Max_iteration);

    // Write final state
    writeGridToFile(outputFile);

    // Close output file
    fclose(outputFile);

    if (iter >= Max_iteration) {
        printf("Failed to converge after %d iterations\n", Max_iteration);
    } else {
        printf("Converged after %d iterations\n", iter);
    }

    return 0;
}
```

Code For Gauss Seidel iteration Method:

```
#include <stdio.h>
#include <stdlib.h>
#include <math.h>

#define L 1.0 // length of the domain (x and y)
#define delta_x 0.01 // length of the cell size in x axis
#define delta_y 0.01 // length of the cell size in y axis
#define MX ((int)(L / delta_x) + 1) // number of cells in x axis
#define NY ((int)(L / delta_y) + 1) // number of cells in y axis
#define ERROR 0.0001 // error for the convergence
#define T_max 10.0 // maximum temperature at the boundary condition
#define T_min 0.0 // minimum temperature at the boundary condition
#define Max_iteration 10000 // maximum iteration allowed for solving the equation

double T[MX][NY]; // Temperature grid
double T_new[MX][NY]; // New temperature grid

// Function to set boundary conditions
void setBoundaryConditions() {
    int i, j;
    // Top side Boundary condition
    for (i = 0; i < MX; i++) {
        T[i][NY - 1] = T_max;
    }
    // Bottom side Boundary condition
    for (i = 0; i < MX; i++) {
        T[i][0] = T_max;
```

```
    }
    // Left side Boundary condition
    for (j = 0; j < NY; j++) {
        T[0][j] = T_min;
    }
    // Right side Boundary condition
    for (j = 0; j < NY; j++) {
        T[MX - 1][j] = T_min;
    }
}

// Function to perform Gauss-Seidel iteration
void gaussSeidelIteration(double *max_diff) {
    int i, j;
    double q1 = 1.0, q2 = -1.0;
    int sx1 = (int)(0.45 / delta_x), sy1 = (int)(0.5 / delta_y);
    int sx2 = (int)(0.55 / delta_x), sy2 = (int)(0.5 / delta_y);

    *max_diff = 0.0; // Reset max_diff

    for (i = 1; i < MX - 1; i++) {
        for (j = 1; j < NY - 1; j++) {
            double source = 0;
            if (i == sx1 && j == sy1) source = q1;
            if (i == sx2 && j == sy2) source = q2;

            // Update T[i][j] using the most recent values
```

```
            double old_T = T[i][j];
            T_new[i][j] = 0.25 * (T[i + 1][j] + T[i - 1][j] + T[i][j + 1] + T[i][j - 1] -
(delta_x * delta_x) * source);

            // Calculate error
            double error = fabs(T_new[i][j] - old_T);
            if (error > *max_diff) {
                *max_diff = error;
            }

            // Update T[i][j] immediately for Gauss-Seidel
            T[i][j] = T_new[i][j];
        }
    }
}

// Function to write grid to file
void writeGridToFile(FILE *file) {
    for (int i = 0; i < MX; i++) {
        for (int j = 0; j < NY; j++) {
            fprintf(file, "%f ", T[i][j]);
        }
        fprintf(file, "\n");
    }
    fprintf(file, "\n\n"); // Two blank lines between datasets for gnuplot
}

int main() {
```

```
double max_diff;
int iter = 0;

printf("Number of cells in the X-axis: %d\n", MX);
printf("Number of cells in the Y-axis: %d\n", NY);

// Initialize temperature array
for (int i = 0; i < MX; i++) {
    for (int j = 0; j < NY; j++) {
        T[i][j] = 0.0; // Initialize all temperatures to 0
    }
}

// Set boundary conditions
setBoundaryConditions();

// Open output file
FILE *outputFile = fopen("temperature_field.txt", "w");
if (outputFile == NULL) {
    printf("Error opening output file!\n");
    return 1;
}

// Gauss-Seidel iteration
do {
    gaussSeidelIteration(&max_diff);
```

```
        iter++;

        // Write grid to file every iteration
        writeGridToFile(outputFile);

    } while (max_diff > ERROR && iter < Max_iteration);

    // Write final state
    writeGridToFile(outputFile);

    // Close output file
    fclose(outputFile);

    if (iter >= Max_iteration) {
        printf("Failed to converge after %d iterations\n", Max_iteration);
    } else {
        printf("Converged after %d iterations\n", iter);
    }

    return 0;
}
```

Code For successive under relaxation method iteration Method:

```
#include <stdio.h>
#include <stdlib.h>
#include <math.h>

#define L 1.0 // length of the domain (x and y)
#define delta_x 0.01 // length of the cell size in x axis
#define delta_y 0.01 // length of the cell size in y axis
#define MX ((int)(L / delta_x) + 1) // number of cells in x axis
#define NY ((int)(L / delta_y) + 1) // number of cells in y axis
#define ERROR 0.0001 // error for the convergence
#define T_max 10.0 // maximum temperature at the boundary condition
#define T_min 0.0 // minimum temperature at the boundary condition
#define Max_iteration 10000 // maximum iteration allowed for solving the equation
#define alpha 0.7 // Under-relaxation factor

double T[MX][NY]; // Temperature grid
double T_new[MX][NY]; // New temperature grid

// Function to set boundary conditions
void setBoundaryConditions() {
    int i, j;
    // Top side Boundary condition
    for (i = 0; i < MX; i++) {
        T[i][NY - 1] = T_max;
    }
    // Bottom side Boundary condition
```

```
    for (i = 0; i < MX; i++) {
        T[i][0] = T_max;
    }
    // Left side Boundary condition
    for (j = 0; j < NY; j++) {
        T[0][j] = T_min;
    }
    // Right side Boundary condition
    for (j = 0; j < NY; j++) {
        T[MX - 1][j] = T_min;
    }
}

// Function to perform Successive Under-Relaxation iteration
void surIteration(double *max_diff) {
    int i, j;
    double q1 = 1.0, q2 = -1.0;
    int sx1 = (int)(0.45 / delta_x), sy1 = (int)(0.5 / delta_y);
    int sx2 = (int)(0.55 / delta_x), sy2 = (int)(0.5 / delta_y);

    *max_diff = 0.0; // Reset max_diff

    for (i = 1; i < MX - 1; i++) {
        for (j = 1; j < NY - 1; j++) {
            double source = 0;
            if (i == sx1 && j == sy1) source = q1;
            if (i == sx2 && j == sy2) source = q2;
```

```
            // Update T[i][j] using the most recent values
            double old_T = T[i][j];
            T_new[i][j] = 0.25 * (T[i + 1][j] + T[i - 1][j] + T[i][j + 1] + T[i][j - 1] -
(delta_x * delta_x) * source);

            // Apply under-relaxation
            T[i][j] = old_T + alpha * (T_new[i][j] - old_T);

            // Calculate error
            double error = fabs(T_new[i][j] - old_T);
            if (error > *max_diff) {
                *max_diff = error;
            }
        }
    }
}

// Function to write grid to file
void writeGridToFile(FILE *file) {
    for (int i = 0; i < MX; i++) {
        for (int j = 0; j < NY; j++) {
            fprintf(file, "%f ", T[i][j]);
        }
        fprintf(file, "\n");
    }
    fprintf(file, "\n\n"); // Two blank lines between datasets for gnuplot
}
```

```
int main() {
    double max_diff;
    int iter = 0;

    printf("Number of cells in the X-axis: %d\n", MX);
    printf("Number of cells in the Y-axis: %d\n", NY);

    // Initialize temperature array
    for (int i = 0; i < MX; i++) {
        for (int j = 0; j < NY; j++) {
            T[i][j] = 0.0; // Initialize all temperatures to 0
        }
    }

    // Set boundary conditions
    setBoundaryConditions();

    // Open output file
    FILE *outputFile = fopen("temperature_field.txt", "w");
    if (outputFile == NULL) {
        printf("Error opening output file!\n");
        return 1;
    }

    // Successive Under-Relaxation iteration
    do {
```

```
        surIteration(&max_diff);

        iter++;

        // Write grid to file every iteration
        writeGridToFile(outputFile);

    } while (max_diff > ERROR && iter < Max_iteration);

    // Write final state
    writeGridToFile(outputFile);

    // Close output file
    fclose(outputFile);

    if (iter >= Max_iteration) {
        printf("Failed to converge after %d iterations\n", Max_iteration);
    } else {
        printf("Converged after %d iterations\n", iter);
    }

    return 0;
}
```